

INTERFACE GRÁFICA COM O USUÁRIO – PARTE 2

COMPONENTES

5. JRadioButton – os botões de opção são semelhantes a caixas de seleção no sentido de que têm dois estados - selecionado e não-selecionado. Entretanto, os RadioButtons aparecem como um grupo em que apenas um botão de opção pode ser selecionado por vez. Selecionar um botão automaticamente força todos os outros botões no grupo a serem removidos da seleção. O relacionamento lógico entre os botões de opção é mantido por um objeto ButtonGroup. Precisamos agrupar os botões de rádio adicionando os mesmos em um componente ButtonGroup.

Abaixo temos um exemplo de aplicação que tem quatro botões de rádio (Negrito, Itálico, Normal, Negrito e Itálico) e quando clicamos nos botões de rádio ele troca a letra do jTextField2 para o estilo correspondente:

```
public class TelaPrincipal extends javax.swing.JFrame {
    private Font Fnormal, Fnegrito, Fitalico, FnegritoItalico;

    public TelaPrincipal() {
        initComponents();
        jLabel1.setToolTipText("Este é o Label1");
        RadioButtonHandler rbh = new RadioButtonHandler();
        rbn.addItemListener(rbh);
        rbi.addItemListener(rbh);
        rbnor.addItemListener(rbh);
        rbni.addItemListener(rbh);

        radioGroup = new ButtonGroup();
        radioGroup.add(rbn);
        radioGroup.add(rbi);
        radioGroup.add(rbnor);
        radioGroup.add(rbni);
        Fnormal = new Font("TimesRoman",Font.PLAIN,14);
        Fnegrito = new Font("TimesRoman",Font.BOLD,14);
        Fitalico = new Font("TimesRoman",Font.ITALIC,14);
        FnegritoItalico = new Font("TimesRoman",Font.BOLD+Font.ITALIC,14);
    }

    private class RadioButtonHandler implements ItemListener {
        public void itemStateChanged(ItemEvent e){
            if (e.getSource()==rbn){
                jTextField2.setFont(Fnegrito);
            }
            if (e.getSource()==rbi){
                jTextField2.setFont(Fitalico);
            }
            if (e.getSource()==rbnor){
                jTextField2.setFont(Fnormal);
            }
            if (e.getSource()==rbni){
                jTextField2.setFont(FnegritoItalico);
            }
            jTextField2.repaint();
        }
    }
}
```

6. JComboBox – é também chamada de caixa de combinação (também chamada de lista drop-down) fornece uma lista de itens entre os quais o usuário pode escolher.

O exemplo abaixo utiliza um JComboBox para fornecer uma lista de quatro nome de arquivos de imagem. Quando um nome de arquivo de imagem é selecionado, a imagem correspondente é exibida como um Icon em um JLabel.

```
public class TesteJComboBox extends javax.swing.JFrame {
    private String nomes[] = {"casa.jpg", "pessoa.jpg",
                              "aviao.jpg", "cao.jpg"};
    private ImageIcon icones[] = {new ImageIcon(nomes[0]),
                                  new ImageIcon(nomes[1]),
                                  new ImageIcon(nomes[2]),
                                  new ImageIcon(nomes[3])};

    public TesteJComboBox() {
        initComponents();
        DefaultComboBoxModel cbm = new DefaultComboBoxModel();
        for(int i = 0; i < nomes.length; i++){
            cbm.addElement(nomes[i]);
        }
        jcb.setModel(cbm);
        jcb.addItemListener(new ItemListener(){
            public void itemStateChanged(ItemEvent e){
                jLabel1.setIcon(icones[jcb.getSelectedIndex()]);
            }
        });
    }
}
```

7. JList – exibe uma série de itens em que o usuário pode selecionar um ou mais desses itens.

Exemplo abaixo: cria um Jlist de 13 cores e quando o nome de uma cor for selecionada através de um clique na JList ocorre um ListSelectionEvent e a cor de fundo da janela altera-se.

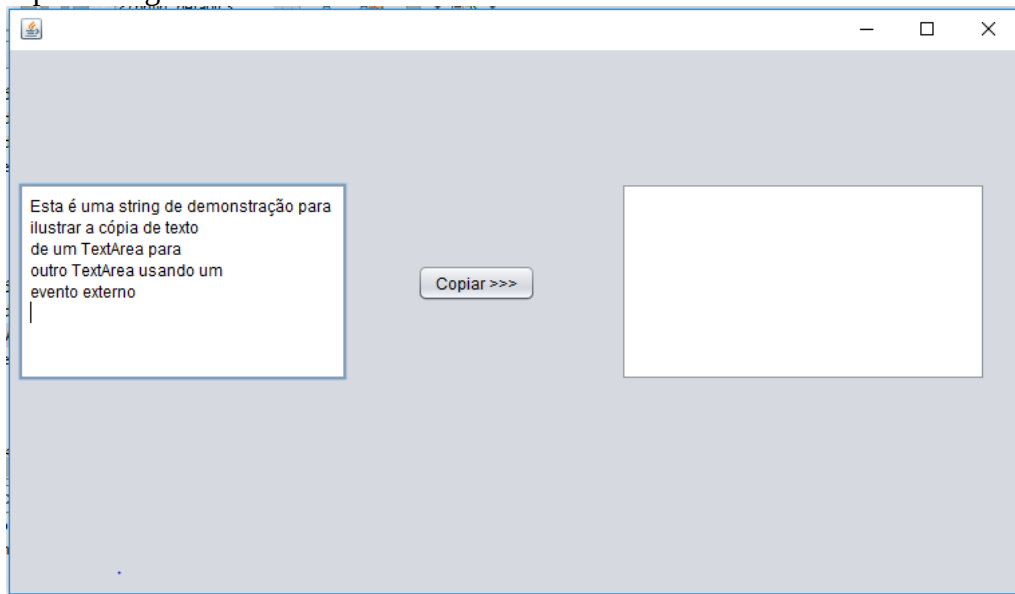
```
public class TesteJList extends javax.swing.JFrame {
    private String nomescores[] = {"Preto", "Azul", "Ciano",
        "Cinza escuro", "Cinza", "Cinza claro", "Verde", "Magenta",
        "Laranja", "Rosa", "Vermelho", "Branco", "Amarelo"};
    private Color cores[] = {Color.BLACK, Color.BLUE, Color.CYAN,
        Color.DARK_GRAY, Color.GRAY, Color.LIGHT_GRAY, Color.GREEN,
        Color.MAGENTA, Color.ORANGE, Color.PINK, Color.RED, Color.WHITE,
        Color.YELLOW};
    private Container c;

    public TesteJList() {
        initComponents();
        c = getContentPane();
        DefaultListModel lm = new DefaultListModel();
        for(int i = 0; i < nomescores.length; i++){
            lm.addElement(nomescores[i]);
        }
        jList1.setModel(lm);
        jList1.setVisibleRowCount(5); //o número de elementos visíveis na lista
        //tratamento dos eventos

        jList1.addListSelectionListener(
            new ListSelectionListener(){
                public void valueChanged(ListSelectionEvent e){
                    c.setBackground(cores[jList1.getSelectedIndex()]);
                }
            }
        );
    }
}
```

8. JTextArea – fornece uma área para a digitação de várias linhas de texto.

No exemplo abaixo temos dois JTextArea em que será digitado um texto na primeira área de texto e quando selecionada uma parte do texto e clicado no botão Copiar a aplicação copiará o texto selecionado para a segunda área de texto.



```
public class TesteAreaTexto extends javax.swing.JFrame {

    public TesteAreaTexto() {
        initComponents();
        String s = "Esta é uma string de demonstração para \n"+
            "ilustrar a cópia de texto\n" +
            "de um TextArea para\n"+
            "outro TextArea usando um\n"+
            "evento externo\n";
        jTextArea1.setText(s);
    }

    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        jTextArea2.setText(jTextArea1.getSelectedText());
    }
}
```

Obs: Para colocar uma string como conteúdo de um JTextArea devemos usar o método append() do componente JTextArea:

Exemplo:

```
jTextArea1.append("Nome: "+nome
                + "\nSexo:" + sexo);
```